

SNIA SDC: StorageAI 2026 - AiSIO: Orchestrating Storage I/O Across CPUs and Accelerators

메타데이터

항목	값
채널	SNIAVideo
업로드	2026-05-12
재생 시간	29분 34초
URL	https://www.youtube.com/watch?v=AJe22Nah5KA
자막 출처	manual (en)

핵심 요약

Samsung 연구원이 AiSIO(Accelerator-integrated Storage I/O, 또는 ACIO)를 발표한다. NVMe 스토리지 I/O가 호스트 DRAM을 경유하지 않고 GPU 가속기 메모리를 직접 목표로 삼을 수 있도록 Linux 스토리지 스택을 확장하는 오픈소스 프로젝트다. 강연은 세 부분으로 구성된다. 첫째, 현재 가속기 I/O를 제약하는 세 가지 소프트웨어 병목 진단. 둘째, libvm 기반 초기 프로토타입을 대체하는 새 아키텍처(HOMIE, Extend Access Library, Ublock 인프라 중심) 소개. 셋째, Samsung Memory Research Center에 설치된 Dell 서버(NVMe 16개)를 대상으로 한 벤치마크 결과. 애플리케이션·드라이버 레이어 최적화를 통해 6,200만 IOPS에 필요한 CPU 비용을 8코어에서 1.5코어 상당으로 줄였으며, GPU 메모리를 대상으로 하는 피어-투-피어 DMA가 호스트 메모리 대상과 동일한 NVMe 처리 비용을 가진다는 것을 보였다.

컴포넌트 개요 발표 중 청중인 Kristoff가 F_MAP_AP 기반 접근이 디버깅 도구 수준이며 동시 접근 시 손상 위험이 있다고 강하게 이의를 제기했고, Q&A에서 이를 재확인했다. 발표자는 이의를 인정하면서 PNF와 flex files도 관리 레이어 후보로 검토 중이라고 답했다.

다루는 주제

도입 및 문제 정의 (00:00:05-00:03:43)

강연은 AiSIO를 세 범주의 소프트웨어 오버헤드를 중심으로 위치시키는 것으로 시작한다. 설계 목표는 대체가 아닌 상호운용성(interoperability over replacement), 즉 기존 스토리지 스택을 확장하되 대체하지 않는 것으로 명시된다.

- 세 가지 과제: (1) 애플리케이션, 라이브러리, 언어 런타임, 유저/커널 경계, OS 스토리지 스택 각 레이어의 오버헤드; (2) 가속기에 도달하기 전 호스트 DRAM을 경유하는 원치 않는 데이터 복사; (3) 스토리지 장치의 PCIe 단일 물리 함수 제한.
- 지원하는 세 가지 I/O 모드: CPU 개시(데이터를 호스트 메모리로), CPU 개시 피어-투-피어(CPU가 주도하며 데이터는 가속기 메모리로), 장치 개시 피어-투-피어(가속기가 직접 개시하며 데이터를 자신의 메모리로).
- 설계 원칙: 기존 스택을 확장하되 기존 추상화와 성능을 희생하지 않는다.
- 모든 컴포넌트가 Linux 커널 업스트림에 이미 포함되었거나 업스트림을 목표로 한다.
- 비전: 가속기가 스토리지 스택의 일급 시민(first-class citizens)이 되는 것.

관련 연구 및 초기 프로토타입 (00:03:43-00:08:22)

발표자는 기존 솔루션을 검토한 뒤, BAM과 libenvm의 선행 연구를 재현하고 파일 시스템 지원을 추가한 첫 번째 프로토타입을 설명한다.

- 인용된 기존 솔루션: Nvidia GDS, BAM(Big Accelerator Memory), Nvidia Scattera, AMD ROCK XIO, libenvm.
- Nvidia GDS 벤치마크: 단일 스레드 소규모 I/O에서 NVMe 장치 한 대도 포화시키지 못했다.
- 초기 프로토타입은 SIOV(드라이브 하나에 다중 물리 함수)를 활용해 커널 스택 드라이버와 ACIO 프로토타입 드라이버를 동시에 결합했다.
- 파일 익스텐트 캐시를 구성해 GPU가 파일 시스템 조회 없이 직접 LBA를 로드할 수 있게 했다.
- 블록 I/O 결과: ACIO 프로토타입이 BAM과 동등; Nvidia GDS는 특히 96 스레드 환경에서 성능 저하를 보였다.
- 소규모 및 대규모 파일 워크로드에 걸친 파일 처리량 비교에서 소규모 파일에서의 이점이 확인됐다.
- 프로토타입의 과제: 오픈소스 스택과의 코드베이스 호환성, 업스트림 가능성, 파일 익스텐트 조회 신뢰성.

신규 AiSIO 아키텍처 (00:08:22-00:11:46)

새 아키텍처는 libvm, 커스텀 커널 패치, 독점적 장치 소유권을 데몬 기반 제어 플레인과 표준 커널 인터페이스로 대체한다.

- 새 아키텍처는 libvm, 커스텀 커널 패치, 독점적 장치 소유권을 제거한다.
- HOMIE(Host Orchestrated Multipath IO Daemon): 제어 플레인, 호스트에서 모든 I/O 경로를 초기화한다.
- UDMA buff import: GPU 물리 메모리 주소에 대한 접근을 제공한다.
- Extend Access Library: HOMIE 내부에 파일 익스텐트 정보를 캐시; XFS 온디스크 포맷 직접 디코딩을 커널의 F_MAP_AP 인터페이스로 대체했다.
- 유저 스페이스 프로세스는 UIO PCI generic 및 VFIO PCI를 통해 HOMIE에 접근; 소규모 ioctl이 DMA buff 및 UDMA buff 인터페이스를 연결한다.
- SIOV 또는 다중 물리 함수를 지원하는 장치: 하드웨어 보조 다중 드라이버 결합; 단일 함수 장치: Linux Ublock 인프라로 소프트웨어 기반 유저 스페이스 제어 플레인 구현.
- Extend Access Library에 F_MAP_AP 결과와 함께 파일 변경 알림을 처리하는 EBPF 트레이스 추가를 계획 중이다.

오픈소스 컴포넌트 (00:11:46-00:16:19)

기존 프로젝트는 파란색, 신규 또는 확장 항목은 주황색으로 표시된 컴포넌트 다이어그램 발표 중에 Kristoff의 이의 제기가 있었고, 발표자는 나머지 도구 소개를 이어갔다.

- XNME: AiSIO 프리미티브로 확장된 크로스 플랫폼 I/O 라이브러리; XNME가 통합된 어느 곳에서든 새 I/O 경로를 사용할 수 있다.
- XME Perf: 최소 오버헤드로 CPU 개시 및 장치 개시 I/O 패턴을 측정하는 벤치마크 도구.
- UPC: CPU와 가속기 양쪽에서 사용 가능한 MMIO 도어벨 모듈; PCIe 도어벨 호출을 추상화해 드라이버 개발을 간소화한다.
- DMA buff 인프라 탭 모듈: 아직 업스트림에 포함되지 않음; Linux 커널 커뮤니티 피드백을 위해 공개했다.
- Extend Access Library: XFS 원시 디코딩 → F_MAP_AP 백엔드 → (계획 중) F_MAP_AP + EBPF 조합 순으로 진화했다.
- Field Path: POSIX, GDS CUFIL, AiSIO 인터페이스를 파일 수준에서 비교하는 벤치마크 도구.
- CHO: Ansible 유사 프로비저닝 도구; 전체 AiSIO 스택의 재현 가능한 배포를 지원한다.
- 백서와 SDK가 공개되어 있다.

벤치마크 결과 (00:16:19-00:25:20)

네 가지 실험: 튜닝된 CPU 기준, 애플리케이션 레이어 최적화, 드라이버 레이어 최적화, 피어-투-피어 GPU 대상 I/O(CPU 개시 및 장치 개시), 그리고 대역폭 포화 분석.

- 테스트 환경: Samsung Memory Research Center에 설치된 Dell 서버, NVMe 드라이브 16개, 비할당 모드(NVME LBA format); 드라이브당 비할당 IOPS: 3.8백만.
- SPDK BDEV Perf 기준: CPU 8코어로 NVMe 16개를 포화시켜 6,200만 IOPS; 코어당 7.7백만 IOPS로 선형 스케일링; NUMA 효과로 일부 변동.
- 애플리케이션 레이어 비교(동일 NVME 드라이브, 프런트엔드 상이): BDEV Perf — 하드웨어 스레드 1/2/3개에서 630만/740만/1,600만 IOPS; BDEV 레이어 제거(NVME Perf) 시 극적인 향상; XNME를 통한 XME Perf로 하드웨어 스레드 3개에서 3,200만 IOPS 달성.
- UPCE 툴링을 이용한 드라이버 레이어 최적화: 하드웨어 스레드 1개에서 약 4,000만 IOPS; CPU 1개/하드웨어 스레드 2개에서 약 5,000만 IOPS; 이 시점에서 병목은 장치 수이며 소프트웨어가 아니다.
- 전체 CPU 효율 향상: 6,200만 IOPS에 8코어 → 1.5코어 상당으로 감소.
- CPU 개시 피어-투-피어(GPU로 데이터 전송): 호스트 메모리 기준과 동일한 IOPS 상한; 피어-투-피어 DMA를 위한 추가 CPU 비용 없음.
- 장치 개시 피어-투-피어: 구현 확인; 동일한 IOPS 상한 달성; 이 모드의 GPU 자원 소모량은 슬라이드에 미보고.
- 피어-투-피어 대역폭: 4K I/O 크기에서 NVMe 드라이브 3개가 PCIe Gen 5 GPU 링크를 포화; 4번째 드라이브 추가 시 이점 미미; PCIe 프로토콜 오버헤드가 제한 요인으로 식별.
- 4K I/O 크기에서 CPU 한 개로 GPU 8개를 대역폭 기준으로 포화시킬 수 있다.

결론 및 Q&A (00:25:20-00:29:31)

발표자가 비전을 요약하고 청중 질문을 받는다; Kristoff가 이전 이의 제기를 Q&A에서 더 상세히 전달한다.

- 모든 컴포넌트가 오픈소스이며 이미 업스트림에 있거나 업스트림을 목표로 한다.
- 장치 개시 실험의 새 결과가 추가로 공개되어 있다.
- 소스 코드와 백서가 공개되어 있다.
- Kristoff의 Q&A 이의 제기: F_MAP_AP는 파일 시스템별로 내용이 다른 디버깅 도구이며, 동시 접근 시 손상 위험이 크다; 2년 전에 팀과 관리자에게 이 문제를 제기하고 PFS 블록 레이아웃을 재배치 메커니즘 포함 방식으로 확장할 것을 권고했다고 밝혔다.

- 발표자 답변: 팀에서 PNF와 flex files의 장점도 검토 중이며 F_MAP_AP만 사용하는 것이 아니라고 설명했다.
- 벤치마크 파일: 사전 할당된 고정 크기.
- 데이터 로더 워크로드: 이미지 데이터셋(TikTok 데이터셋 포함)과 8 GB 파일을 GPU에서 무작위 접근; Nvidia DALI 프레임워크에서 영감을 받아 설계했다.

인물 · 조직 · 제품 · 장소

인물

- Kristoff (강연 중 F_MAP_AP 접근 방식에 이의를 제기한 청중)

조직

- Samsung
- Nvidia
- AMD
- Dell
- SNIAMVideo (채널)

제품

- AiSIO / ACIO (Accelerator-integrated Storage I/O)
- HOMIE (Host Orchestrated Multipath IO Daemon)
- Nvidia GDS (GPU Direct Storage)
- BAM (Big Accelerator Memory)
- Nvidia Scattera
- AMD ROCK XIO
- SPDK (BDEV Perf, NVME Perf)
- XNME
- XME Perf
- UPC
- UPCE
- Field Path
- CHO
- Extend Access Library
- Unblock
- libenvm
- Nvidia DALI

장소

- Samsung Memory Research Center (Dell 벤치마크 서버 설치 장소)

수치 및 데이터

수치	설명
3.8백만 IOPS	드라이브당 비할당 모드(NVME LBA format) IOPS
6,200만 IOPS	NVMe 16개, CPU 8코어 기준(SPDK BDEV Perf)
7.7백만 IOPS	CPU 코어당 기준 IOPS (선형 스케일링)
630만 IOPS	BDEV Perf, 하드웨어 스레드 1개
740만 IOPS	BDEV Perf, 하드웨어 스레드 2개
1,600만 IOPS	BDEV Perf, CPU 코어 2개 위 하드웨어 스레드 3개
3,200만 IOPS	XME Perf, 하드웨어 스레드 3개 (애플리케이션 레이어 최적화)
약 4,000만 IOPS	드라이버 레이어 최적화(UPCE), 하드웨어 스레드 1개
약 5,000만 IOPS	드라이버 레이어 최적화(UPCE), CPU 1개 / 하드웨어 스레드 2개
1.5코어 상당	최적화 후 6,200만 IOPS 유지에 필요한 CPU 자원
NVMe 드라이브 3개	4K I/O 크기에서 PCIe Gen 5 GPU 링크 포화에 필요한 수량
4K	PCIe Gen 5 GPU 링크를 드라이브 3개로 포화시키는 I/O 크기
GPU 8개	4K I/O에서 CPU 한 개로 포화시킬 수 있는 GPU 수(대역폭 기준)
8 GB	데이터 로더 벤치마크에서 사용된 대형 사전 할당 파일 크기

주요 인용

basically we want to extend the existing software stack and we want to do so in ways where we're not trading off the utilities of having existing abstractions and we don't want to trade off performance either. — unknown

that's the vision. We want to be able to have accelerators as uh uh first class citizens in the um in the storage stack. — unknown

a single thread small IO we cannot saturate even a single NVME device using this. — unknown

we hit the 62 million IOPS here with eight CPU cores. So that's 16 devices. Uh that gets saturated by using eight CPU cores. — unknown

with three hardware threads you get 32 million IOPS. — unknown

from these eight cores to re reach 62 million down to 1.5 — unknown

the NVME drive doesn't really care that the DMA address is on your host memory or on your GPU. The cost of processing that IO is the same. — unknown

is a proper way to do it and I told for two years to multiple members in your team how to do it and it's really upsetting that you can keep publishing kind of dangerous — Kristoff

debugging tool. It has different content for different file systems. It's a massive risk for uh cost and corruption due to concurrent activity. That's why I told everyone including your boss two years ago, you need to expand the PFS block layout layouts that have all the mechanisms to deal with that including relocation mechanism mechanism for defending clients — Kristoff

that was more a comment than a than a question and uh I think we'll take that into consideration. — unknown